Claude Code Essentials
Cheat Sheets

These cheat sheets are provided for personal and non-commercial use only.

Please do not redistribute or upload these cheat sheets elsewhere.

Good luck on your exam!

Claude Code Essentials CheatSheet

Exam Pro

Agentic Coding Tool (aka Coding Agent) that **reads your codebase, edits files, runs commands**, and integrates with your development tools.
Coding Agents can be available in your terminal, IDE, desktop app, and browser.
Code Harness (Agentic Harness) is a system that wraps an LLM with prompts, tools, and execution environments to turn it into an agent that can take actions and run workflows.
Claude Code is an **agentic coding tool**. It that reads your codebase, edits files, runs commands, and integrates with your development tools.

Agentic Loop

A continuous cycle where an LLM gathers context, takes actions using tools, and verifies results until the goal is achieved.

- Core loop: **Gather Context → Take Action → Verify Results → repeat**
- Uses **tool calls** (functions/APIs) to interact with code, systems, and external services
- **Self-reflects** after each step to decide next action
- Stops only when the **goal is met**
- You can **interrupt or guide** the loop at any time
- Tools enable real actions: read/edit files, run commands, call APIs, test code

Agentic Loop – Models

You can **choose the model** used throughout the agentic loop
Model choice impacts **speed, cost, and reasoning ability**
Key models:

- **default** → auto-selected, recommended
- **sonnet** → balanced, best for most coding tasks
- **opus** → strongest reasoning, slower, higher cost
- **haiku** → fastest, cheapest, for simple tasks

- **sonnet[1m]** → large context (long sessions)
- **opusplan** → opus for planning, sonnet for execution

Core idea:

Pick the model based on **task complexity vs cost/speed tradeoff**

Claude API Key can be used to control cost and usage for Claude Code.

Useful for **production-use-cases and automated systems.**

The token is generated at: https://platform.claude.com

Claude Code – API Key via Third Party

- Claude can be accessed via **third-party providers** (Bedrock, Vertex AI, Azure AI Foundry)
- You **authenticate with the provider's API key**, not directly with Claude

API Token Usage – Credits

- Buy credits to use the API
- Tier determines **purchase limits**
- More usage → higher tier
- Credits **expire in 1 year**

Subscription

- Fixed plan covers **Claude usage (Code + Cowork)**
- Usage tracked by **session + weekly limits**
- Model choice ≠ token pricing (no multipliers shown)

Extra Usage

- Can enable **pay-as-you-go beyond limits**
- Set **spend limits** to control costs

Claude Code Essentials CheatSheet

Exam Pro

The **context window** is the **total "working memory" a model can hold** at once it includes both its input and output measured in tokens.

Context Window Size

- Model's **working memory (tokens)** = input + output
- Acts as a **length limit per conversation**
- Larger window → more context, fewer drops
- Exceeding limit → **older content trimmed/summarized**

Usage vs Length Limits

- **Usage limits** = how much you can use over time (budget)
- **Length limits** = how much fits in one chat (context window)
- All surfaces count toward **same usage limit**

Current session shows how much of your **5-hour session limit** you've used and the **time remaining**.

Claude Code Session is functionality a **conversation to the Claude code agent**.

- You can resume a previous session by using the **/resume** command.

Forking Sessions allow you to resume a previous session without affecting the original session.

/context shows **tokens consumed in the current session** and available tokens, broken down by category

Installing Claude Code

- Linux/Mac/WSL: curl -fsSL https://claude.ai/install.sh | bash
- Windows: irm https://claude.ai/install.ps1 | iex

Claude **auth login** or **/login** to get authenticated.

We can use **claude auth status** to check if we are authenticated

- Diagnose and verify your Claude Code installation with **claude doctor** or **/doctor**
- **/stats** to view usage pattern
- The **/usage** command will show you information for your Subscription uages within the Claude CLI tool.

Claude Code Essentials *CheatSheet*

Exam Pro

ccusage analyzes **Claude token usage** from local JSONL logs

- Shows **cost, usage, and expensive conversations**
- Helps compare **subscription vs pay-per-use value**
- Install/run: npx ccusage@latest

Claude Modes

- **Normal (Auto)** → Executes tasks directly, asks permission when needed
- **Plan Mode** → Generates a plan first for review before execution

The **/cost** command will show you API Token usage.

/status command will let us know what **method of authentication** we are using (are we logged in?)

/logout and check your **Env Vars**

/btw command allows you **ask a question in parallel** while you have a request currently in progress.

/voice command allows use to **use your voice to send prompts**

/export command lets you **export the current conversation** to the **clipboard** or a **file**.

/compact – create a summarized conversation to save token.

/clear – clear the current conversation. Clear will not clear out CLAUDE.me files or AutoMemory

Interactive Mode - Real-time **CLI interaction loop** with Claude. Immediate feedback + continuous prompting

Key shortcuts:

- **Ctrl+C** → cancel
- **Ctrl+D** → exit
- **Ctrl+L** → clear screen
- **Ctrl+R** → search history
- **Ctrl+T** → toggle tasks
- **Shift+Tab** → toggle permissions

Claude Code Essentials CheatSheet

Exam Pro

Claude Code Surfaces

- "Surface" = **where you interact** with Claude
- Examples: CLI, Web, Desktop, VS Code, JetBrains, CI/CD, Slack
- Most surfaces require **subscription or API access**

Claude Code on Desktop is a native desktop app for your OS.
Claude Code on Web provides a sandbox developer environment with the Claude CLI via a web interface. Claud Code on Web directly connects to GitHub Repos.
Claude Chrome Extension can be used to **operate your web-browser**
to perform any natural task you would normally do in your browser.
Claude Code Visual Studio Extension provides friendly interface over the CLI terminal within VSCode.
JetBrains IDE allows you to install a plugin so you can use Claude CLI directly.
Claude Code on GitHub Actions allows us to **trigger Claude Code** based on workflows to perform tasks in repo.
Remote Control connects claude.ai/code or the Claude app for iOS and Android to a Claude Code session running on your machine
You can use **/cost** to get a cost of usage for API usage. For **subscription** you need to monitor the cost on your subscription page.
The **status line** is a **customizable bar** at the bottom of Claude Code that runs any shell script you configure.
/statusline command can be used to generate out the code to display a custom status line.
You can use natural language to tell it remove the statusline e.g. **/statusline** clear **/statusline** remove it
~/.claude/settings.json is where statusline is configured.
Claude Code sends the following **JSON Sessions data** as STDIN to the script for **Statusline**.

Claude Code Projects

- Running claude in a folder creates a **project**
- Stored under .claude/projects
- Project name = **based on directory path**

- Contains **sessions + memory**

Session Files

- Each session = **.jsonl file (event log)**
- Each line = **event (user, system, file snapshot, etc.)**
- Includes metadata: **timestamp, cwd, sessionId, etc.**

The /**security-review** command lets you **run security analysis** directly from your terminal before committing code.

Claude Code development container for teams that need consistent, secure environments.

Claude Code Settings Scope

- Settings defined via settings.json at multiple levels
- **Managed** → org-wide (highest priority)
- **User** → global personal (~/.claude/settings.json)
- **Project** → shared in repo (.claude/settings.json)
- **Local** → personal, not in git (settings.local.json)

Claude Code Settings

🔐 **Auth**

- API keys + AWS auth helpers
- Force login method / org

🗂 **Sessions**

- Auto-delete old sessions
- Memory + plans storage paths

🌍 **Env**

- Global variables for all runs
- (secrets, configs, flags)

Claude Code Essentials CheatSheet

Exam Pro

🤖 **Models**

- Set default model
- Limit available models
- Map to providers (e.g. Bedrock)

🗣️ **Output**

- Style (e.g. explanatory)
- Language
- Optional git instructions

📃 **Git**

- Commit / PR attribution
- Co-author toggle

📢 **Updates**

- Startup announcements
- Update channel (latest / stable)

⏳ **UI**

- Show duration
- Customize spinner text
- Tips + progress bar
- Reduce motion

👥 **Agents**

- Teammate mode (auto | tmux | inline)

🔒 **Permissions**

- allow → auto run

Claude Code Essentials CheatSheet

Exam Pro

- ask → confirm
- deny → block
- extra directories + default mode

🔌 **MCP**

- Enable/disable servers
- Allowlist / denylist

🧩 **Plugins (Managed)**

- Allow/block marketplaces
- Custom trust message

🔒 **Lockdown**

- Managed-only → no user/project overrides

🪝 **Hooks**

- On/off all hooks
- Limit URLs + env vars
- Run commands after actions

📊 **Misc**

- OTel headers
- Status line
- @file autocomplete
- Respect .gitignore

Permissions Rules

- Levels: **allow | ask | deny**
- Order: **deny → ask → allow** (most restrictive wins)

Claude Code Essentials CheatSheet

- Tools: Bash, Read/Write, WebFetch, MCP, Agents

Bash Wildcard Permissions

- Wildcard: * matches any characters. Works anywhere in command
- Spaces matter (ls * only)

Read & Edit Rules

- Follows .gitignore patterns
- // absolute path
- ~/ home directory
- / project root
- none = current directory

WebFetch Rules

- Controlled by domain
- WebFetch(domain:example.com)
- Can allow / ask / deny per domain

MCP Rules

- mcp__server = all tools
- mcp__server__tool = specific tool
- Supports wildcards

Bare Tool Rules: No () = full access

- Examples: Read, Bash, WebFetch
- Very broad permissions

Claude Code Essentials *CheatSheet*

Exam Pro

Tools

- Agent → subagents
- Bash* → run commands
- Edit* → modify files
- Read → read files
- WebFetch* → fetch URLs
- WebSearch* → search web
- Write* → create/overwrite files
- Grep / Glob → search & match
- TaskCreate / TaskGet / TaskList, TaskUpdate / TaskStop / TaskOutput

Permission Modes for tool approval:

- **default** Standard behavior: prompts for permission on first use of each tool
- **acceptEdits** Automatically accepts file edit permissions for the session
- **plan** Plan Mode: Claude can analyze but not modify files or execute commands
- **dontAsk** Auto-denies tools unless pre-approved via /permissions or permissions.allow rules
- **bypassPermissions** Skips all permission prompts (requires safe environment, see warning below)

--dangerously-skip-permission is a setting which will tell Claude for a session to **stop prompting you to ask for permissions**:

A **sandbox** is a **security mechanism for separating running programs** for your OS.

A sandbox provides a controlled environment for programs to run with:

- storage and memory scratch space
- limited network access
- restricted ability to inspect the host system

Claude Code Essentials CheatSheet

Exam Pro

Model Configuration

- Set default model in settings.json
  - "model": "opus"
- Start with a model:
  - claude --model opus
- Switch during session:
  - /model sonnet
- Restrict models:
  - "availableModels": ["sonnet", "haiku"]

Headless Tasks Permissions - You can use **–allowTools and --disallowTools** to apply permissions for that specific headless task.

When you press **ESC key while a task is running** you can interrupt the an **active task**.

When you **press ESC twice** it will **clear your current prompt** you have typed.

Claude Code Image Reasoning - Claude Code can take **images as inputs** to analyze an image or use it as guidance for generating out code.

Effort Command

Controls response depth vs token usage

/effort [low | medium | high | max | auto]

- low → fastest, cheapest (simple tasks)
- medium → balanced
- high → complex reasoning (default-like)
- max → deepest reasoning (Opus only)
- auto → Claude decides

/rename lets you rename the current session.

Claude Code Essentials CheatSheet

Exam Pro

/rewind restore a session to an older point.
/debug mode will output verbose information to **a specific text file**.
Claude will also prompt you to describe the problem as it attempts to investigate the issue.
When Claude finishes working and is waiting for your input, it **fires a notification events**.

Terminal Notifications

- Fires when Claude finishes or needs input
- Mac/Linux: works by default
- Windows/WSL2: use hooks

Notification Hooks

- Custom handlers for notification events
- Used when no native notifications

Triggers:

- permission_prompt → needs approval
- idle_prompt → done
- elicitation_dialog → needs input
- auth_success → auth complete
- Can run scripts (e.g. send alerts)

Hooks are user-defined shell commands, HTTP endpoints, or LLM prompts that execute automatically at specific points in Claude Code's lifecycle.
How to troubleshoot hooks:

- Turn on Debug Mode –debug / debug and then tail the transcripts for hooks fired..
- Check /hooks to see if Claude see its registered

/simplify will review changed code for reuse, quality, and efficiency, then fix any found issues.

Claude Code Essentials CheatSheet

Exam Pro

Fast mode is a high-speed configuration for Claude Opus 4.6, making the model 2.5x faster at a higher cost per token.
Agent skills are a lightweight, **open format** for extending AI agents. This open format we created by Anthropic in 2025
Skills use **progressive disclosure** to manage context efficiently:

- **Discovery**: At startup, agents load only the name and description of each available skill, just enough to know when it might be relevant.
- **Activation**: When a task matches a skill's description, the agent reads the full SKILL.md instructions into context.
- **Execution**: The agent follows the instructions, optionally loading referenced files or executing bundled code as needed.

A **SKILLS.md** requires a minimum a frontmatter header with **name** and **description**
Markdown body contains the **actual instructions,** structured however you like.
At minimum, only the SKILL.md file is **required** to create a skill.
Skills are **activated/triggered** by agent based on the **contents of the skill's description**.

Agent Skills – Activation

- Triggered by skill **description match**
- Loads name/description first → full skill on match
- Only used if agent **needs help**

Be specific (not too vague/detailed), Write as: *"Use when…",* Focus on user goal, not internals, Include keywords + symptoms
A test case has three parts:

- Prompt: a realistic user message — the kind of thing someone would actually type.
- Expected output: a human-readable description of what success looks like.
- Input files (optional): files the skill needs to work with.

Store test cases in **evals/evals.json** inside your skill directory:

Claude Code Essentials CheatSheet

Exam Pro

Skills can execute external code/programs in two ways:

- one-off commands
- **self-contained scripts**

Claude skills are Agent Skills. They follow the Agent Skills open format.

Claude Skills

- Skills = **specialized workflows** (agent skills format)
- Can run **automatically (match)** or **via command**
- Built-in skills exist but **don't show in /skills**

Skill Types & Location

- Types: **Reference (knowledge)** | **Task (actions)**
- Locations: **Personal (~/.claude/...) | Project (.claude/...)**

Dynamic Content

- Inject runtime data with !command
- Keeps skills **up-to-date (PRs, logs, etc.)**

Context Fork (Isolation)

- context: fork → runs in **subagent (isolated)**
- Use for **deep research / heavy tasks**

Permissions & Control

- Skills can auto-use tools via allowed-tools
- Disable all → **deny Skill tool**
- Allow/deny specific: Skill(name *)

Scheduling: Setup up **cron scheduling tools** to run prompts repeatedly, poll for status, or set one-time reminders within a Claude Code session.

Claude Code Essentials CheatSheet

Exam Pro

Code Review analyzes your GitHub pull requests and posts findings as inline comments on the lines of code where it found issues.
Subagents are specialized AI assistants that handle specific types of tasks.
Claude will delegate tasks to agent **based on agents description**.

Subagents

- Specialized agents for **specific tasks**
- Auto-selected based on **description match**
- Run **in-session only**, report to main agent

Key Behavior

- Each subagent has:
  - **Own context**
  - **Own tools**
  - **Own permissions**

Built-in

- **Explore** → read/search code (no changes)
- **Plan** → gathers context before planning
- **General-purpose** → complex multi-step tasks
- **Bash** → runs terminal commands

Custom Subagents

- Stored in ~/.claude/agents/
- Defined via **markdown frontmatter**

Claude Code Essentials CheatSheet

Exam Pro

Must know fields:

- **name** → trigger
- **description** → when it runs (MOST important)
- **tools / disallowedTools**
- **model**
- **permissionMode**

Advanced Controls

- maxTurns → limits execution
- isolation → separate worktree
- hooks → run logic before/after tools

Agent Teams

- Coordinate **multiple Claude instances** working together
- One **team lead** + multiple **teammates**
- Teammates have **isolated context** but can **communicate**

Subagents vs Teams

- **Subagents** → quick, isolated workers (report back only)
- **Agent Teams** → collaborative, share findings + coordinate

When to Use

- Parallel exploration (different ideas/approaches)
- Separate features or layers (frontend/backend/tests)
- Same problem, **different strategies**

Key Behavior

- Teammates **work independently**
- Share via **task list + communication**
- Can interact with **any teammate directly**

Enable (Important)

- Disabled by default
- Set ENV:
- CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1

Structure

- Team Lead
- Teammates
- Shared Task List
- Mailbox (communication)

Plugins are **packages for Claude Code** which allow to **share a bundle skills, agents, hooks, and MCP servers** across projects and teams.

- The **plugin.json** defines the plugin

Language Server Protocol (LSP) is protocol between a code editor and tools that understand code (language servers).

Language Server powers the editing experience for programming language.

It implements features such as:

- Autocomplete
- error-checking (diagnostics)
- jump-to-definition
- many other language features

Claude Code Essentials CheatSheet

Exam Pro

The **Model Context Protocol (MCP)** is an open standard for **connecting AI agents to external systems**.
MCP server has the following:

- Tools – tools that an external model can call
- Resources – data that an external model can ready
- Prompts – reusable prompt an external model can use.

MCP Config

- Add **custom MCP servers** via config
- Used for **private APIs, DBs, internal tools**

LSP with Plugins

- Add .lsp.json to support **new languages**
- Connects Claude to a **language server**

The official Anthropic Plugin Marketplace lets you find plugins for Claude Code.

MCP Environments

- **Local (stdio)** → spawned on demand, auto-managed
- **Production (HTTP/SSE)** → always running, URL-based

Agent SDK allows you to **run Claude Code programmatically** from the **CLI, Python, or TypeScript.**
Agent SDK enables you to build your own agents directly powered by Claude Code.
Output Styles controls **how Claude responds**

- Claude Code has three **types of built-in Output Style**:
  - Default / Explanatory / Learning

Custom Styles

- Stored in ~/.claude/output-styles
- **Overrides system prompt**

Claude Code Essentials CheatSheet

Claude Code can connect to hundreds of external tools and data sources through the Model Context Protocol (MCP)

- **Adding** an MCP server connection to Claude (added to **.claude.json**):

${VAR} - Expands to the value of environment variable VAR

${VAR:-default} - Expands to VAR if set, otherwise uses default

- **command** - The server executable path
- **args** - Command-line arguments
- **env** - Environment variables passed to the server
- **url** - For HTTP server types
- **headers** - For HTTP server authentication

~/.claude.json is a legacy/alternative user-level configuration file for Claude Code

Playwright is an **end-to-end (2d2) testing framework** to **simulate real user interactions across an entire web app.**

- Microsoft's **Playwright MCP server** allows an agent to interact with a Playwright server so agents can automatically test.

Persistent Context are **markdown files** that will **automatically load into every new context.**

Persistent Context occurs in Claude in two ways:

- 1. User-defined markdown files e.g. Claude.md
- 2. Agent-defined markdown files Auto Memory

CLAUDE.md

- Loaded into **every new session (system prompt level)**
- Acts as **persistent instructions/context**

Scope Levels (priority matters)

- Org → Project → User → Local
- Lower level = **more specific override**

Claude Code Essentials CheatSheet

Exam Pro

File Locations

- **Project:** ./CLAUDE.md or ./.claude/CLAUDE.md
- **User:** ~/.claude/CLAUDE.md
- **Local:** ./CLAUDE.local.md (not committed)

Load Behavior

- Root CLAUDE.md = **fully loaded**
- Subdirectory CLAUDE.md = **lazy loaded (on demand)**

Key Rules

- Keep **< 200 lines**
- Be **specific (not vague)**
- Markdown structure improves adherence

Imports

- Use @file.md to split large configs
- Max depth: **5 levels**

Exclusions

- claudeMdExcludes → ignore unwanted CLAUDE.md files
- Useful for **monorepos / legacy rules**

/init → auto-generates CLAUDE.md from codebase

The top level CLAUDE.md file will go into the **.claude/** directory.

Claude Rules allows you **organize instructions into multiple files** using **for larger projects.**

Claude Auto Memory will pay attention to your preferences and automatically remember them.

Claude remembers by **making notes** (plain markdown file).

- ON by default

Claude Code Essentials ***CheatSheet***

Exam Pro

- **Machine-local only** (not shared across devices)
- **Per-project memory** (shared across same repo/worktrees)

Each project gets its own memory directory at **~/.claude/projects/<project>/memory/**

MEMORY.md acts as an index of the memory directory.

- Claude **reads + writes memory during sessions**
- "Writing memory" / "Recalled memory" = active updates
- /memory command → view/manage memory

Claude Imports vs Claude Rules vs AutoMemory

@imports in CLAUDE.md are for ***documentation and context****, they provide guidance and understanding to Claude but are not hard rules.*

/rules are **rules**, rules that Claude should absolutely follow.

AutoMemory is Claude's best guesses of trying to figure how it should work.

Troubleshooting Memory

- Claude isn't following my CLAUDE.md → check **/memory**, file location, specificity
- Don't see memory → /memory → open folder
- Too large → keep **<200 lines**, split files
- Lost after /compact → file wasn't saved properly